

**Matricola:** \_\_\_\_\_

**Cognome** \_\_\_\_\_ **Nome:** \_\_\_\_\_

1) Si illustri la complessità computazionale della convoluzione di un'immagine con un filtro e si descrivano brevemente una o più tecniche per migliorare l'efficienza di tale operazione.

2) Definire formalmente l'operatore di apertura della morfologia matematica e illustrarne brevemente il funzionamento.

[illegible]

3) Descrivere lo spazio colore RGB e illustrarne brevemente vantaggi e svantaggi.

[illegible]

4) Utilizzando NumPy e OpenCV, implementare in Python la funzione *esercizio(img)* che riceve un'immagine grayscale *img* (con un byte per pixel) e deve eseguire le seguenti operazioni:

- 1) Calcolare, per ogni pixel dell'immagine *img*, la media dei pixel in un intorno 7x7. Impostare a zero 3 pixel lungo tutti i bordi nell'immagine risultato.
- 2) Calcolare l'immagine *Diff* in cui ciascun pixel è il valore assoluto della differenza fra il pixel corrispondente nell'immagine originale e il valore determinato al passo precedente.
- 3) Eseguire l'operazione di contrast stretching su *Diff*.
- 4) Binarizzare *Diff* utilizzando come soglia globale il valore 128.
- 5) Etichettare le componenti connesse del risultato ottenuto al passo precedente; eliminare quindi le componenti connesse con area inferiore a 15 pixel.
- 6) Restituire come output un'immagine grayscale in cui i pixel appartenenti alle componenti connesse rimanenti dopo il passo precedente hanno valore 255, mentre tutti gli altri 0.

```
import numpy as np
import cv2 as cv

def esercizio(img):
```

*Promemoria: alcune funzioni che potrebbero essere utili*

```
cv.threshold(src, thresh, maxval, type) -> retval, dst
cv.boxFilter(src, ddepth, ksize) -> dst
connectedComponentsWithStats(image) -> retval, labels, stats, centroids
```

*Promemoria: alcune costanti che potrebbero essere utili*

```
cv.THRESH_BINARY
cv.CC_STAT_AREA
```